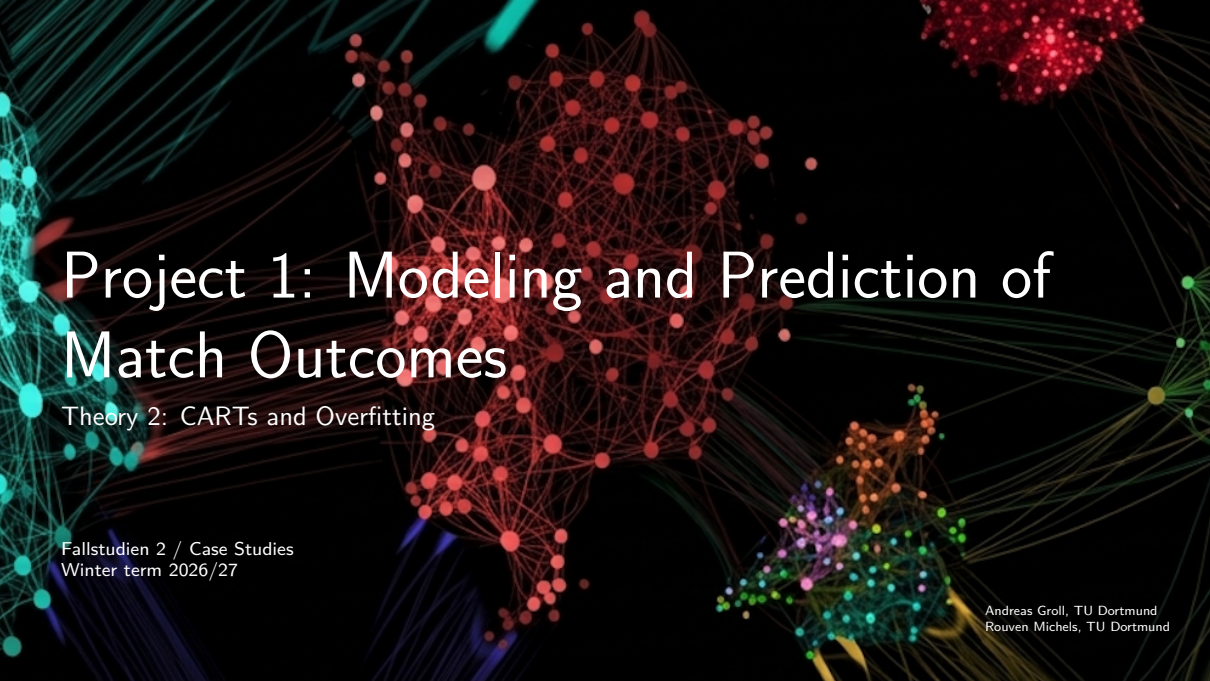


Project 1: Modeling and Prediction of Match Outcomes

Theory 2: CARTs and Overfitting

Fallstudien 2 / Case Studies
Winter term 2026/27

Andreas Groll, TU Dortmund
Rouven Michels, TU Dortmund

Acknowledgement

Big thanks goes to Prof. Dr. Bernd Bischl and Julia Moosbauer from the Department of Statistics at Ludwig-Maximilians-University Munich for providing parts of these slides.

Introduction

Introduction

Classification and Regression Trees (CART)

- introduced by Breiman et al. (1984),
- same basic structure for both classification and regression,
- binary splits are constructed top-down,
- constant prediction in each terminal node (leaf): either a **numerical value**, a **class label**, or a **probability vector** over class labels.

Tree model and prediction

Motivation for tree models:

- trees can easily capture **non-linearity**,
- they can also easily capture (potentially complex) **interactions**,
- they implicitly account for **variable selection**,
- no **distributional assumption** (compared to regression)
- finally, they are easy to interpret via a **dendrogram**.

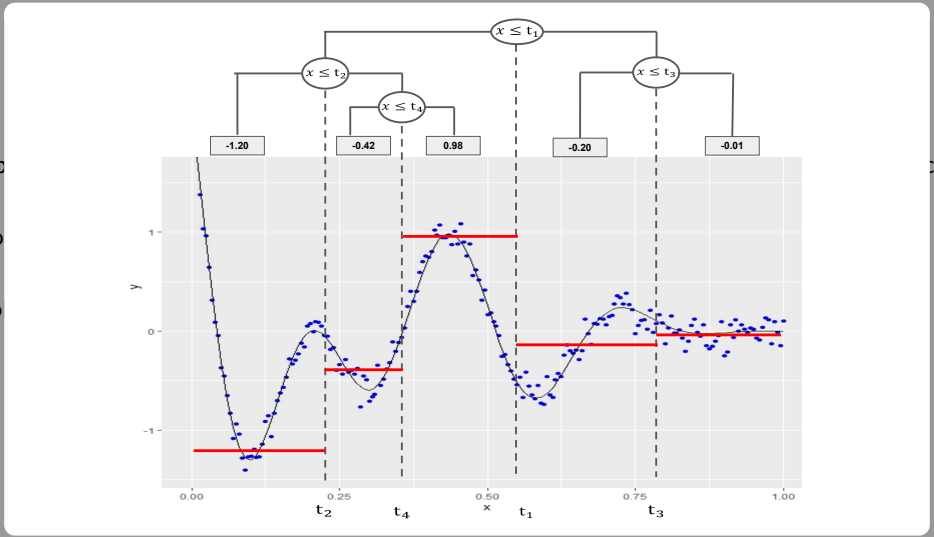
Tree model and prediction

- for predictions, observations are passed down the tree, according to splitting rules in each node,
- an observation will end up in **exactly one leaf node**,
- all observations in a leaf node are assigned the **same prediction** for the target.

Tree model and prediction

- for p
- an o
- all o

ch node,

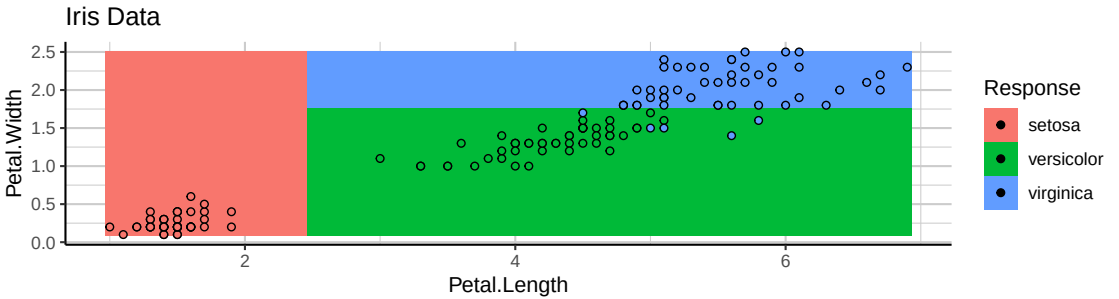


Trees as an additive model

- Each point in the feature space $\mathcal{X}$ is assigned to exactly one leaf, and each leaf has a set of input points leading to it through axis-parallel splits.
- Hence, for metric features, trees divide the **feature space $\mathcal{X}$** into M **rectangular regions**.
- In each leaf node (region) the tree model yields the predicted (constant) numerical response (i.e., the average), class label or class distribution.
- Note that some additional **mathematical details** can be found in the appendix.

Trees as an additive model

- Each point in the feature space $\mathcal{X}$ is assigned to exactly one leaf, and each leaf has a set of input points leading to it through axis-parallel splits.
- Hence, for metric features, trees divide the **feature space $\mathcal{X}$** into M **rectangular regions**.
- In each leaf node (region) the tree model yields the predicted (constant) numerical response (i.e., the average), class label or class distribution.
- Note that some additional **mathematical details** can be found in the appendix.



Splitting criteria

Splitting criteria

- A split w.r.t. **feature x_j at split point t** divides a parent node $\mathcal{N}$ into two new child nodes $\mathcal{N}_1$ and $\mathcal{N}_2$.
- Mathematically, this means:

$$\mathcal{N}_1 = \{(\mathbf{x}, y) \in \mathcal{N} : x_j \leqslant t\} \text{ and } \mathcal{N}_2 = \{(\mathbf{x}, y) \in \mathcal{N} : x_j > t\}.$$

- In order to evaluate how good a split is, we compute the empirical risks in both child nodes and sum them up.
- Finding the best way to split $\mathcal{N}$ into $\mathcal{N}_1, \mathcal{N}_2$ then means finding the optimal combination of **feature x_j** and corresponding **split point t**, which leads to the minimal risk sum.

Note that again more **mathematical details** can be found in the appendix.

Splitting criteria

Empirical risk minimization

- Let $\mathcal{N}_r \subseteq \mathcal{D}$ be the data that is assigned to a certain node $\mathcal{N}_r$ of a tree.
- Then, the risk $\mathcal{R}(\mathcal{N}_r)$ for this leaf $\mathcal{N}_r$ is defined as the average loss for the data points assigned to that leaf under a given loss function $L(\cdot, \cdot)$:

$$\mathcal{R}(\mathcal{N}_r) = \frac{1}{|\mathcal{N}_r|} \sum_{(\mathbf{x}, y) \in \mathcal{N}_r} L(y, c_r)$$

- The prediction for all points in leaf $\mathcal{N}_r$, i.e. $(\mathbf{x}, y) \in \mathcal{N}_r$, is then simply given by the optimal constant $c_r = \arg\min_{c_r} \mathcal{R}(\mathcal{N}_r)$.

Splitting criteria: Regression

- For regression trees, we usually use L_2 **loss**:

$$\mathcal{R}(\mathcal{N}) = \frac{1}{|\mathcal{N}|} \sum_{(\mathbf{x},y)\in\mathcal{N}} (y - c)^2$$

- One can proof that the best constant prediction within a node $\mathcal{N}$ under L_2 is the **mean**:

$$c = \bar{y}_{\mathcal{N}} = \frac{1}{|\mathcal{N}|} \sum_{(\mathbf{x},y)\in\mathcal{N}} y$$

Splitting criteria: Regression

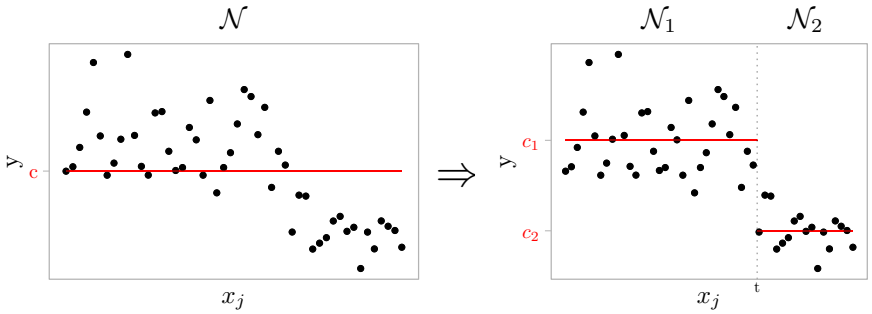
- This means the best split is the one that minimizes the (pooled) variance of the target distribution in the child nodes $\mathcal{N}_1$ and $\mathcal{N}_2$.

  We can also interpret this as a way of measuring the **impurity** of the target distribution, i.e., how much it diverges from a constant in each of the child nodes.

Splitting criteria: Regression

- This
  distr

  We
  how



, i.e.,

Splitting criteria: Regression

- This means the best split is the one that minimizes the (pooled) variance of the target distribution in the child nodes $\mathcal{N}_1$ and $\mathcal{N}_2$.

  We can also interpret this as a way of measuring the **impurity** of the target distribution, i.e., how much it diverges from a constant in each of the child nodes.

- For L_1 **loss**, c is the **median** of $y \in \mathcal{N}$.

Splitting criteria: Classification

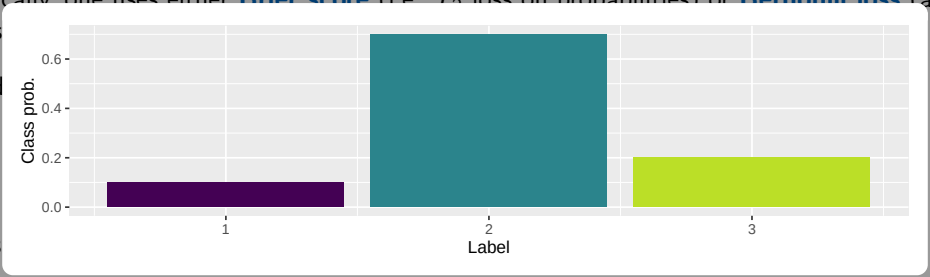
- Typically, one uses either **Brier score** (i.e., L_2 loss on probabilities) or **Bernoulli loss** (as in logistic regression) as loss functions.
- Predicted probabilities in node $\mathcal{N}$ for any class k are simply the class proportions in the node:

$$\hat{\pi}_k^{\mathcal{N}} = \frac{1}{|\mathcal{N}|} \sum_{(\mathbf{x},y)\in\mathcal{N}} \mathbb{1}(y = k)$$

This is the optimal prediction under both the logistic / Bernoulli loss and Brier loss.

Splitting criteria: Classification

- Typically, one uses either **Brier score** (i.e., L_2 loss on probabilities) or **Bernoulli loss** (as in logis
- Pred node:



This

Splitting criteria: Comments

- Splitting criteria for trees are usually defined in terms of **impurity reduction**. Instead of minimizing empirical risk in the child nodes over all possible splits, a measure of **impurity** of the distribution of the target y in the child nodes is minimized.
- For regression trees, the **impurity** of a node is usually defined as the variance of the y_i in the node. Minimizing this **variance impurity** is equivalent to minimizing the squared error loss for a predicted constant in the nodes.
- Minimizing the **Brier score** is equivalent to minimizing the **Gini impurity**.
- Minimizing the **Bernoulli loss** is equivalent to minimizing **entropy impurity**.

Tree growing

Tree growing

We start with an empty tree, a root node that contains all the data.

Trees are then grown by recursively applying **greedy** optimization to each node $\mathcal{N}$.

Greedy means we do an **exhaustive search**: all possible splits of $\mathcal{N}$ on all possible points t for all features x_j are compared in terms of their empirical risk.

The training data are then assigned to child nodes according to the optimal split; the procedure is repeated in all child nodes.

Tree growing

Start with a root node including all data, then search for a feature and split point that minimizes the empirical risk in the child nodes.

Nodes display their current label distribution here for illustration.

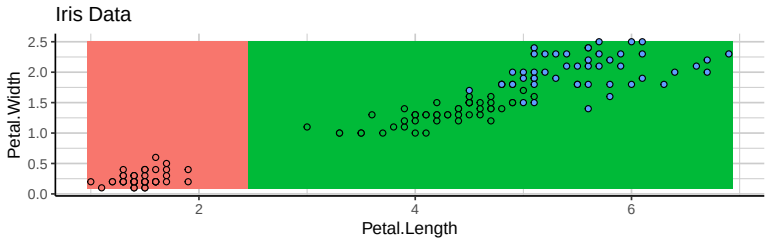
Tree growing

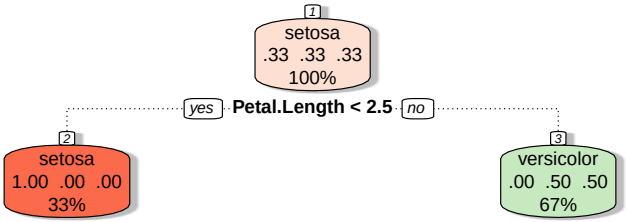
Start with

empirical

Nodes dis

nizes the





Tree growing

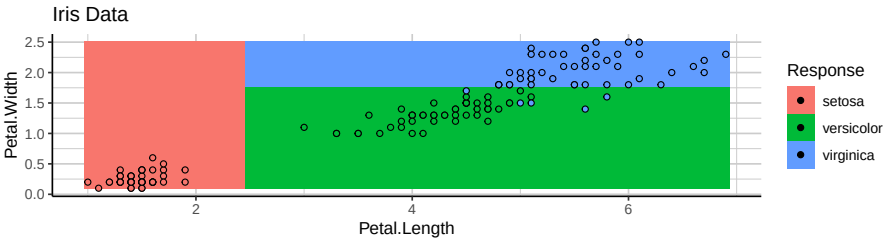
Proceed recursively for each child node:

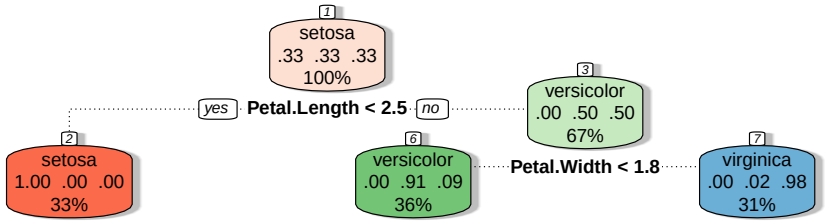
Iterate over all features, and for each feature over all possible split points. Select the best split and divide data in parent node into left and right child nodes:

Tree growing

Proceed r

Iterate ov lit and
divide da

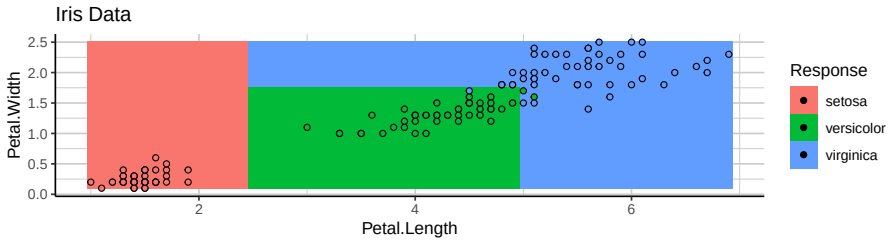


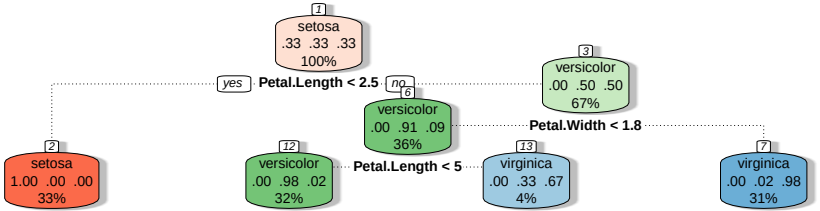


Tree growing

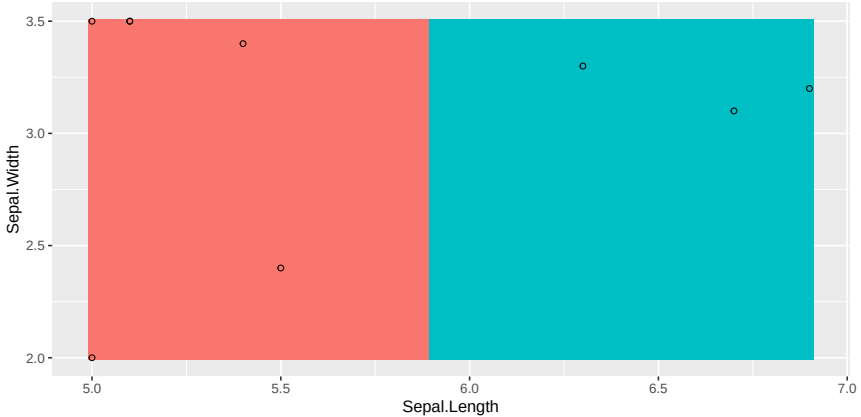
Proceed r

Iterate ov lit and
divide da





Split placement



Splits are usually placed at the mid-point of the observations they split: the large margin to the next closest observations makes better generalization on new, unseen data more likely.

Split improvement

A recent extension and improvement of CARTs are **conditional inference trees**, in short ctrees, and are implemented in the `party` R package (Hothorn et al., 2006; Zeileis et al, 2008).

The following major differences and improvements of ctrees over classical CARTs can be named:

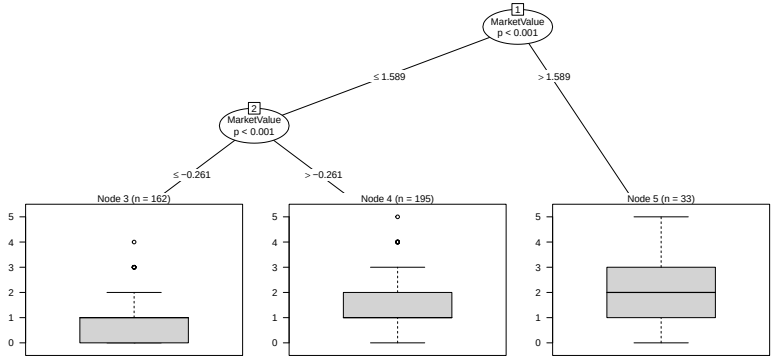
- **Statistical Basis**: Hypothesis testing for splits (vs. minimizing loss function).
- **Overfitting Control**: Built-in cross-validation (vs. post-pruning).
- **Variable Selection**: Adjusted for multiple testing (vs. potential bias in importance).
- **Interpretability**: Clearer insights due to statistical framework (vs. optimization focus).
- **Robustness**: Better with complex interactions vs. (challenges without adjustments).

Example: Football Data

```
library(party)
form.ct <- as.formula(paste("Goals ~ ",paste(colnames(Euro_train)[6:10],collapse="+")))
tree.obj <- ctree(form.ct, data = Euro_train)
plot(tree.obj)
```

Example: Football Data

```
library(
form.ct                                                    +")))
tree.obj
plot(tre
```



Exemplary regression tree for UEFA EURO 2004 – 2020 data. Number of goals is used as response (see boxplots in final nodes), 5 features as described in Chapter 1 are used as predictors.

Stopping Criteria & Pruning

Overfitting Trees

The **recursive partitioning** procedure used to grow a CART would principally run until every leaf only contains a single observation.

- Problem 1: This would take a very long time, as the amount of splits we have to try **grows exponentially** with the number of leaves in the trees.
- Problem 2: At some point before that we should stop splitting nodes into ever smaller child nodes: very complex trees with lots of branches and leaves will **overfit the training data.**
- Problem 3: However, it is very hard to tell where we should stop while we are growing the tree: before we have actually tried all possible additional splits further down a branch, we can't know whether any one of them will be able to reduce the risk by a lot (*horizon effect*).

Stopping Criteria

Problems 1 and 2 can be "solved" by defining different **stopping criteria**:

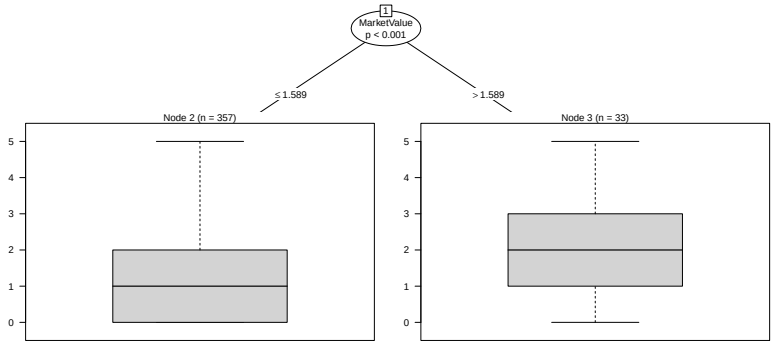
- Stop once the tree has reached a certain number of leaves.
- Don't try to split a node further if it contains too few observations.
- Don't perform a split that results in child nodes with too few observations.
- Don't perform a split unless it achieves a certain minimal improvement of the empirical risk in the child nodes, compared to the empirical risk in the parent node.
- Obviously: Stop once all observations in a node have the same target value (**pure node**) or identical values for all features.

Pruning

We try to solve Problem 3 by **pruning**:

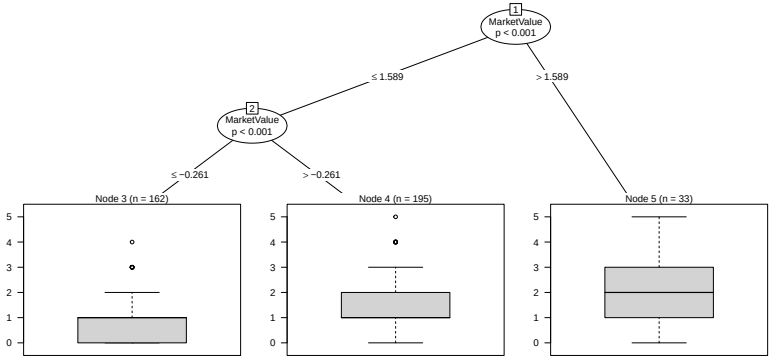
- A method to select the optimal size of a tree.
- Finding a combination of suitable strict stopping criteria ("pre-pruning") is a hard problem: there are many different stopping criteria and it's hard to find the best combination (see chapter on **tuning**).
- Better: Grow a large tree, then remove branches so that the resulting smaller tree has optimal cross-validation risk.
- Feasible without cross-validation: grow a large tree, then remove branches so that the resulting smaller tree has a good balance between training set performance (risk) and complexity (i.e., number of terminal nodes). The trade-off between complexity and accuracy is governed by a **complexity parameter**.
- Such hyperparameters, which control the complexity of a model, are also often called **tuning parameters**, as they need to be tuned (optimized) $\Longrightarrow$ see later.

Pruning



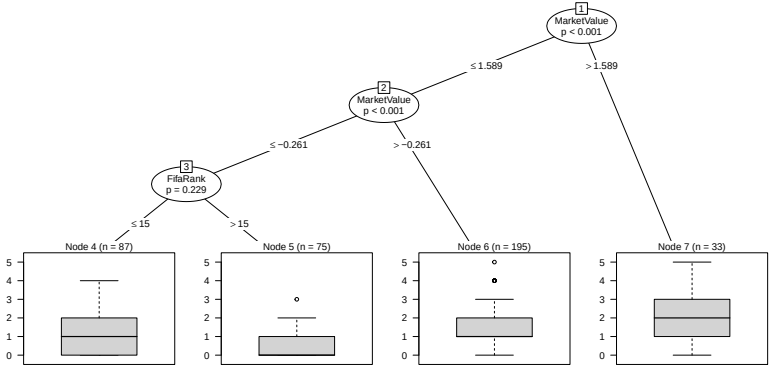
Exemplary regression tree with one split for UEFA EURO 2004 – 2020 data. Number of goals used as response (see boxplots in final nodes), 5 features as described in Chapter 1 are used as predictors.

Pruning



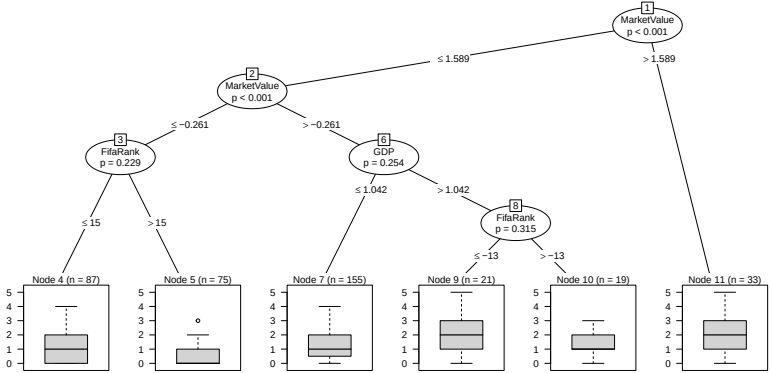
Exemplary regression tree with two splits for UEFA EURO 2004 – 2020 data. Number of goals used as response (see boxplots in final nodes), 5 features as described in Chapter 1 are used as predictors.

Pruning



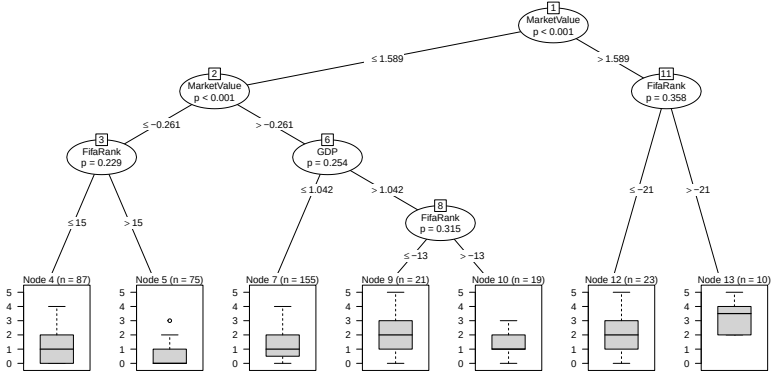
Exemplary regression tree with three splits for UEFA EURO 2004 – 2020 data. Number of goals used as response (see boxplots in final nodes), 5 features as described in Chapter 1 are used as predictors.

Pruning



Exemplary regression tree with five splits for UEFA EURO 2004 – 2020 data. Number of goals used as response (see boxplots in final nodes), 5 features as described in Chapter 1 are used as predictors.

Pruning



Exemplary regression tree with six splits for UEFA EURO 2004 – 2020 data. Number of goals used as response (see boxplots in final nodes), 5 features as described in Chapter 1 are used as predictors.

Capacity & Overfitting

Introduction

- The **capacity** of a statistical or machine learning model (in short, a "learner") represents its flexibility to adjust to the underlying **data generating process**.
- One fundamental goal of statistical and machine learning is to **generalize** on new, unseen data.
- A model's ability to generalize is typically measured by the **generalization error**.
- For this purpose, the data are often split into training and test set; a famous strategy in this regard is **K-fold cross validation (CV)** (see next chapter).
- The capacity of a learner impacts generalization:

  $\Longrightarrow$ too low capacity carries the risk of **underfitting**

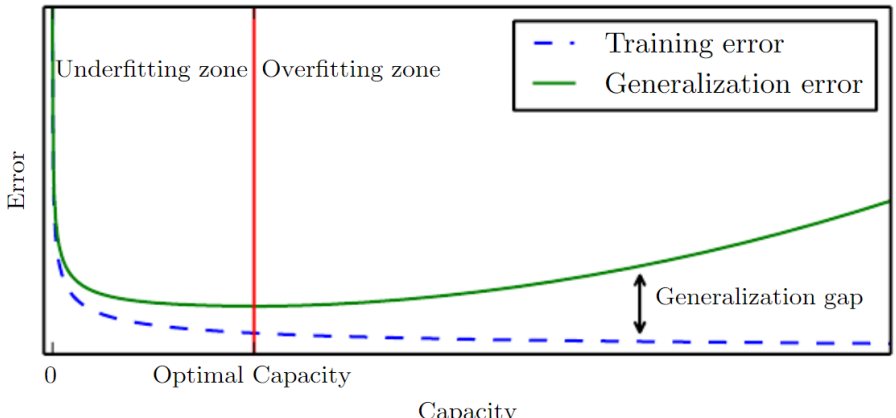
  $\Longrightarrow$ too high capacity carries the risk of **overfitting**

Capacity

- The performance of a learner depends on its ability to:
  - minimize the training error;
  - generalize well to new unseen data ($\Longrightarrow$ low test error).
- Failure to obtain a sufficiently low training error is known as **underfitting**.
- In contrast, if there is a large difference in training and test error, this is known as **overfitting**.

Capacity

- The
  -
  -
- Failu
- In cc



Credit: Ian Goodfellow

fitting.

Capacity

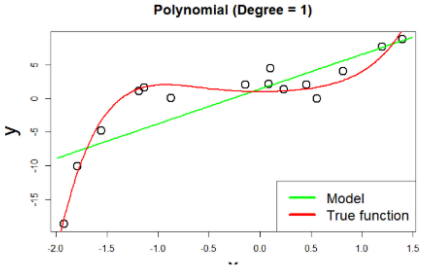
- The performance of a learner depends on its ability to:
  - minimize the training error;
  - generalize well to new unseen data ($\Longrightarrow$ low test error).
- Failure to obtain a sufficiently low training error is known as **underfitting**.
- In contrast, if there is a large difference in training and test error, this is known as **overfitting**.
- Hence, the tendency of a model to over-/underfit is a function of its **capacity**.
- Loosely speaking, a model with low capacity can only learn a few simple relations, whereas a model with large capacity can learn many, possibly complex, relations.
- As the figure shows, the test error is minimized when the model neither underfits nor overfits, i.e., when it has the right capacity.

Capacity

- The capacity (or "complexity") of a model can be increased by extending the size of the corresponding **parameter space**.
- This typically increases the number of parameters to be estimated.
- Examples: increasing the degree of a polynomial effect in linear regression, increasing the depth of a decision tree or a neural network, adding additional predictors, etc.
- As the flexibility of the model increases, the tendency of a model to overfit also increases.
- Such a model might **fit even random noise** in the training data, thereby failing to generalize.

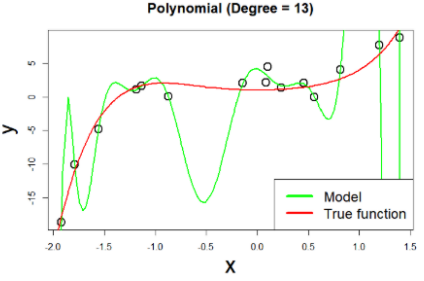
Overfitting: Polynomial Regression

Degree = 1 (highest degree of a term in the polynomial)



Underfitting
(Low Capacity)

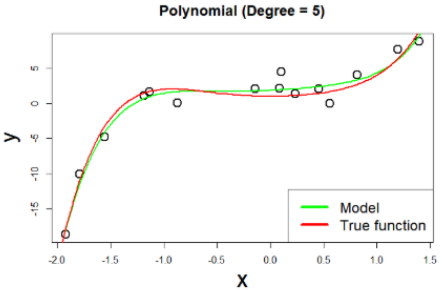
Degree = 13



Overfitting
(High Capacity)

Overfitting: Polynomial Regression

Degree = 5

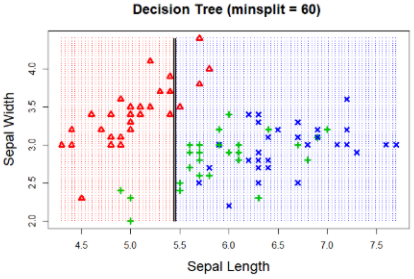


Good fit (Appropriate capacity)

	Degree = 1	Degree = 5	Degree = 13
Training error (RMSE)	3.87	1.23	0.48
Test error (RMSE)	4.11	**1.55**	148.5

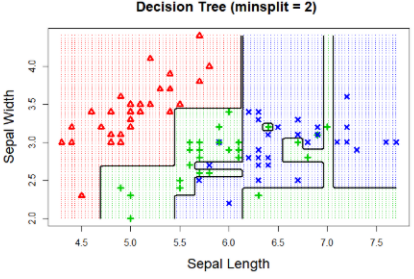
Overfitting: Decision Trees

minsplit = 60 (minimum number of samples in a node being split)



Underfitting (Low Capacity)

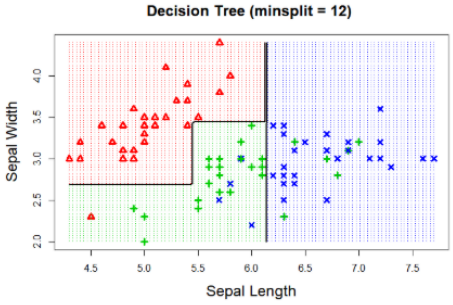
minsplit = 2



Overfitting (High Capacity)

Overfitting: Decision Trees

minsplit = 12

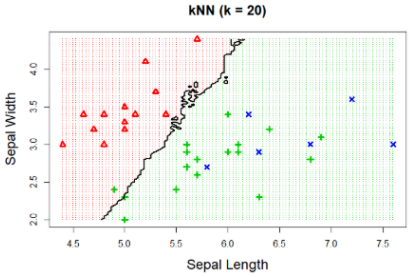


Good fit (Appropriate capacity)

	minsplit = 60	minsplit = 12	minsplit = 2
Training error (Misclassification)	0.36	0.12	0.02
Test error (Misclassification)	0.40	**0.32**	0.35

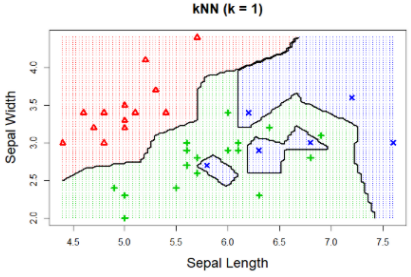
Overfitting: K-Nearest Neighbors

k = 20
(number of neighbours)



Underfitting
(Low Capacity)

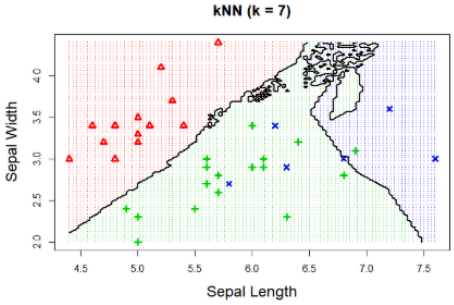
k = 1



Overfitting
(High Capacity)

Overfitting: K-Nearest Neighbors

k = 7



Good fit (Appropriate capacity)

	k = 20	k = 7	k = 1
Training error (Misclassification)	0.22	0.13	0
Test error (Misclassification)	0.40	**0.25**	0.33

Tuning

Many modern statistical and machine learning methods come with certain **tuning parameters**, which control the flexibility of the methods and need to be **optimized**. This is specifically relevant for methods with a **large capacity**.

- **Regression**: add penalization on β_j, e.g. via LASSO $\implies$ penalty strength λ
- **Trees**: complexity of trees (number of layers, minimum node size, etc.)
- **Random Forest**: number of covariates to be randomly sampled and tried out at each split (see next chapter)
- K-**Nearest Neighbors**: number K of neighbors to be considered

Tuning

Many modern statistical and machine learning methods come with certain **tuning parameters**, which control the flexibility of the methods and need to be **optimized**. This is specifically relevant for methods with a **large capacity**.

- **Regression**: add penalization on β_j, e.g. via LASSO $\implies$ penalty strength λ
- **Trees**: complexity of trees (number of layers, minimum node size, etc.)
- **Random Forest**: number of covariates to be randomly sampled and tried out at each split (see next chapter)
- **K-Nearest Neighbors**: number K of neighbors to be considered

To optimize the tuning parameter(s), the data needs to be split (internally, and often repeatedly) into training and test data, e.g. via K-fold CV.

Tuning

Many modern statistical and machine learning methods come with certain **tuning parameters**, which control the flexibility of the methods and need to be **optimized**. This is specifically relevant for methods with a **large capacity**.

- **Regression**: add penalization on β_j, e.g. via LASSO $\implies$ penalty strength λ
- **Trees**: complexity of trees (number of layers, minimum node size, etc.)
- **Random Forest**: number of covariates to be randomly sampled and tried out at each split (see next chapter)
- **K-Nearest Neighbors**: number K of neighbors to be considered

To optimize the tuning parameter(s), the data needs to be split (internally, and often repeatedly) into training and test data, e.g. via K-fold CV.

Finally, the method is fit once again on the full data with the tuning parameter set to the optimum.

Appendix

Trees as an additive model

Each point in the feature space $\mathcal{X}$ is assigned to exactly one leaf, and each leaf has a set of input points leading to it through axis-parallel splits.

Hence, trees divide the feature space $\mathcal{X}$ into **rectangular regions**:

$$f(\mathbf{x}) = \sum_{m=1}^{M} c_m \mathbb{1}(\mathbf{x} \in Q_m)\,,$$

where a tree with M leaf nodes defines M "rectangles" Q_m; moreover, c_m is the predicted numerical response, class label or class distribution in the respective leaf node.

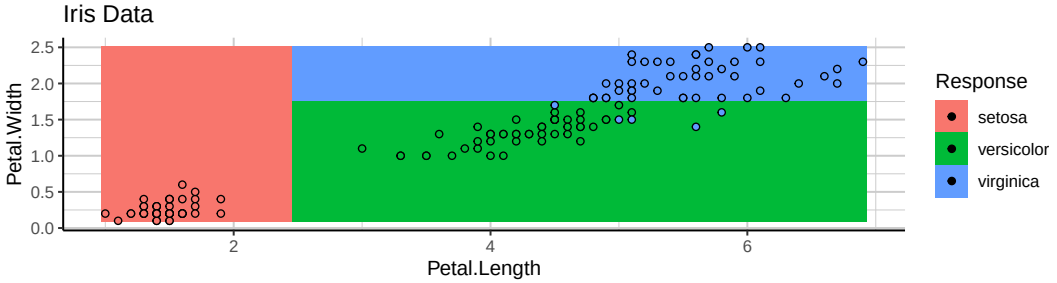
Trees as an additive model

Each point in the feature space $\mathcal{X}$ is assigned to exactly one leaf, and each leaf has a set of input points leading to it through axis-parallel splits.

Hence, trees divide the feature space $\mathcal{X}$ into **rectangular regions**:

$$f(\mathbf{x}) = \sum_{m=1}^{M} c_m \mathbb{1}(\mathbf{x} \in Q_m)\,,$$

where a tree with M leaf nodes defines M "rectangles" Q_m; moreover, c_m is the predicted numerical response, class label or class distribution in the respective leaf node.



Splitting criteria

- A split w.r.t. **feature x_j at split point t** divides a parent node $\mathcal{N}$ into

$$\mathcal{N}_1 = \{(\mathbf{x}, y) \in \mathcal{N} : x_j \leqslant t\} \text{ and } \mathcal{N}_2 = \{(\mathbf{x}, y) \in \mathcal{N} : x_j > t\}.$$

- In order to evaluate how good a split is, we compute the empirical risks in both child nodes and sum them up:

$$\begin{aligned}\mathcal{R}(\mathcal{N}, j, t) &= \frac{|\mathcal{N}_1|}{|\mathcal{N}|}\mathcal{R}(\mathcal{N}_1) + \frac{|\mathcal{N}_2|}{|\mathcal{N}|}\mathcal{R}(\mathcal{N}_2) \\ &= \frac{1}{|\mathcal{N}|}\left(\sum_{(\mathbf{x},y)\in\mathcal{N}_1} L(y, c_1) + \sum_{(\mathbf{x},y)\in\mathcal{N}_2} L(y, c_2)\right)\end{aligned}$$

- Finding the best way to split $\mathcal{N}$ into $\mathcal{N}_1, \mathcal{N}_2$ means solving

$$\arg\min_{j,t} \mathcal{R}(\mathcal{N}, j, t)$$

Splitting criteria: Comments

- Splitting criteria for trees are usually defined in terms of **impurity reduction**. Instead of minimizing empirical risk in the child nodes over all possible splits, a measure of **impurity** of the distribution of the target y in the child nodes is minimized.
- For regression trees, the **impurity** of a node is usually defined as the variance of the y_i in the node. Minimizing this **variance impurity** is equivalent to minimizing the squared error loss for a predicted constant in the nodes.
- Minimizing the **Brier score** is equivalent to minimizing the **Gini impurity**

$$I(\mathcal{N}) = \sum_{k=1}^{K} \hat{\pi}_k^{\mathcal{N}} \left(1 - \hat{\pi}_k^{\mathcal{N}}\right)$$

- Minimizing the **Bernoulli loss** is equivalent to minimizing **entropy impurity**

$$I(\mathcal{N}) = -\sum_{k=1}^{K} \hat{\pi}_k^{\mathcal{N}} \log \hat{\pi}_k^{\mathcal{N}}$$

- Loss functions instead of impurity measures are more straightforward and mathematically equivalent $\implies$ growing a tree can be understood in terms of empirical risk minimization.